UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ciencias
Licenciatura en Ciencias de la Computación

Asignatura: Inteligencia Artificial

Laboratorio 2.12: Recursividad y Espacios de Estados
Tema: Las Torres de Hanoi

Profesora: Dra. Jessica Sarahi Méndez Rincón
Ayudante: Diego Eduardo Peña Villegas
Estudiante: Sita Zoé Sotomayor Carrillo
Fecha: 5 de marzo de 2026

1. Introducción

El problema de las Torres de Hanoi es un ejemplo paradigmático de algoritmo recursivo. El objetivo es mover una pila de n discos de un poste origen a un poste destino, utilizando un poste auxiliar, bajo la restricción de que nunca se puede colocar un disco más grande sobre uno más pequeño. Este reporte analiza el espacio de estados del problema, su complejidad computacional y ofrece una demostración formal del número mínimo de movimientos.

2. Espacio de Estados (Visualización)

El espacio de estados de las Torres de Hanoi forma un grafo no dirigido donde cada nodo representa una configuración válida de los discos y cada arista un movimiento legal. Para n discos, el grafo tiene 3^n nodos y una estructura fractal conocida como el Triángulo de Sierpinski.

A continuación, se visualiza mediante LaTeX (TikZ) el grafo completo para el caso simplificado de $n = 2$ discos (9 estados). Cada nodo indica en qué poste se encuentra cada disco (Disco Grande, Disco Pequeño):

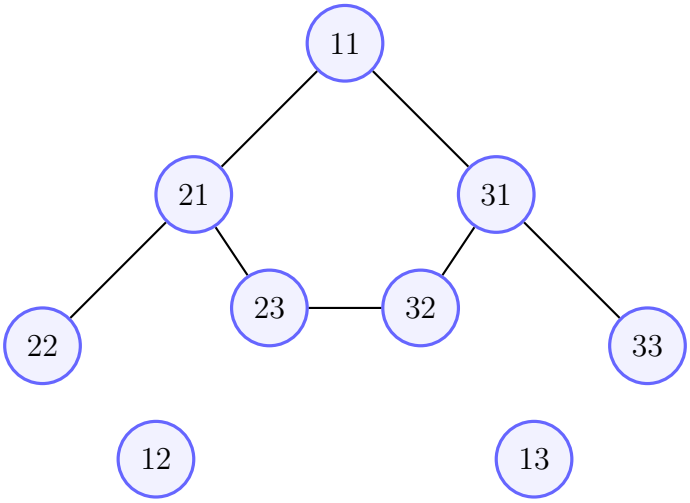

Figura 1: Grafo de Espacio de Estados para $n = 2$. Los nodos representan la posición de los discos.

3. Demostración por Inducción

Se desea demostrar que el número mínimo de movimientos para resolver el problema con n discos es $M(n) = 2^n - 1$.

Base Inductiva ($n = 1$): Para mover 1 disco, se requiere 1 movimiento. Fórmula: $2^1 - 1 = 2 - 1 = 1$. **Se cumple.**

Hipótesis Inductiva: Asumimos que para k discos, $M(k) = 2^k - 1$.

Paso Inductivo ($n = k + 1$): Para mover $k + 1$ discos:

1. Mover k discos al auxiliar: Requiere $M(k)$ movimientos.
2. Mover el disco base $(k + 1)$ al destino: Requiere 1 movimiento.

3. Mover los k discos del auxiliar al destino: Requiere $M(k)$ movimientos.

Total: $M(k+1) = M(k) + 1 + M(k) = 2M(k) + 1$. Sustituyendo la hipótesis:

$$2(2^k - 1) + 1 = 2^{k+1} - 2 + 1 = 2^{k+1} - 1$$

Q.E.D.

4. Análisis de Complejidad

4.1. Complejidad de Tiempo

Observando la recurrencia $T(n) = 2T(n-1) + 1$, el número de pasos crece exponencialmente.

- **Complejidad:** $O(2^n)$.
- **Implicación:** Añadir un solo disco duplica el tiempo de ejecución. Para $n = 64$ (la leyenda de Brahma), el universo se acabaría antes de terminar.

4.2. Complejidad de Espacio

Aunque el tiempo es exponencial, el espacio no lo es. El algoritmo es una búsqueda en profundidad (DFS).

- **Complejidad:** $O(n)$.
- **Justificación:** En cualquier momento dado, la pila de recursión (stack) solo almacena como máximo n llamadas a la función (una por cada disco).

5. Conclusión

La práctica permitió visualizar cómo un algoritmo recursivo simple de tres líneas genera una complejidad de tiempo exponencial. La estructura del espacio de estados demostró ser fractal, conectando la teoría de algoritmos con la geometría. La demostración por inducción valida matemáticamente los resultados empíricos obtenidos en el código.